

Problem 1: Reverse a String

Write a C program that takes a string as input and reverses it without using any library functions such as `strrev()`.

Input:

A single string (without spaces), of length ≤ 100 .

Output:

The reversed version of the input string.

Example:

Input: hello

Output: olleh

Problem 2: Check for Palindrome

Write a C program to check if a given string is a palindrome. A palindrome is a word that reads the same backward as forward.

Input:

A single string (without spaces), of length ≤ 100 .

Output:

Palindrome if the string is a palindrome, otherwise Not Palindrome.

Example:

Input: madam

Output: Palindrome

Input: world

Output: Not Palindrome

Problem 3: Count Vowels, Consonants, Digits, and Spaces

Write a C program to count the number of vowels, consonants, digits, and spaces in a given string. Do not use any functions from `ctype.h`.

Input:

A string of up to 200 characters (may contain spaces, letters, and digits).

Output:

Four integers representing the count of vowels, consonants, digits, and spaces respectively.

Example:

Input: Hello World 123

Output:

Vowels: 3

Consonants: 7

Digits: 3

Spaces: 2

Problem 4: Remove Duplicate Characters

Write a C program to remove all duplicate characters from a string, preserving the order of first appearance.

Input:

A single string (without spaces), of length ≤ 100 .

Output:

The modified string with duplicates removed.

Example:

Input: programming

Output: progamin

Problem 5: Find the Longest Word in a Sentence

Write a C program that finds and prints the longest word in a given sentence. Words are separated by spaces.

Input:

A string of up to 200 characters.

Output:

The longest word. If multiple words have the same maximum length, print the first one.

Example:

Input: The quick brown fox jumps

Output: quick

Problem 6: Check if Two Strings are Anagrams

Write a C program to check if two input strings are anagrams. Two strings are anagrams if they contain the same characters in any order. Assume case-sensitive comparison and no spaces.

Input:

Two strings, each of length ≤ 100 .

Output:

Anagrams if the strings are anagrams, otherwise Not Anagrams.

Example:

Input: listen silent

Output: Anagrams

Input: apple pale

Output: Not Anagrams

Problem 7: Substring Check

Write a C program to check whether one string is a substring of another. Do not use any string library functions.

Input:

Two strings: the first is the main string, the second is the substring to search for.

Output:

Substring Found if the second string is a substring of the first; otherwise, Not Found.

Example:

Input: "hello world", "world"

Output: Substring Found

Problem 8: Remove All Occurrences of a Character

Write a C program that removes all instances of a specified character from a string.

Input:

A string of up to 100 characters and a character to remove.

Output:

The modified string without the specified character.

Example:**Input:** "balloon" "l"**Output:** baoon**Problem 9: Compare Two Strings Without strcmp()**

Write a C program to compare two strings character-by-character to check if they are equal.

Input:

Two strings of length ≤ 100 .

Output:

Equal if the strings match exactly, otherwise Not Equal.

Example:**Input:** "code" "code"**Output:** Equal**Input:** code cope**Output:** Not Equal**Problem 10: Count Occurrences of a Substring**

Write a C program that counts how many times a substring appears in a string. Overlapping matches are allowed.

Input:

Two strings: a main string and a substring to count, both of length ≤ 100 .

Output:

An integer showing how many times the substring occurs in the main string.

Example:**Input:** "ababab" "ab"**Output:** 3